ARYANS

TAKNEEK 2026

# The Big-GEM Theory

**Project:** The Big-GEM Theory
**Event:** Takneek 2026 – Zenith
**Team:** Aryans
**Base project:** NVIDIA GEM GPU RTL simulator
**Main work:** Native GPU support for DSP48E2, CARRY4, and SRLC32E
**Repository:** `https://github.com/Pratham-015/GEM`

## Contents

# 1. Project Summary

## 1.1 Aim of the project

NVIDIA GEM normally converts RTL circuits into one-bit Boolean gates before running them on the GPU. This works well for normal logic, but large hardware blocks lose their original form during synthesis. A multiplier, carry chain, or shift-register LUT can turn into many small gates. This adds more graph nodes and more work for the simulator.

Our project adds a second path for three Xilinx primitives:

- DSP48E2, using the subset fixed by the problem statement;
- CARRY4, including connected carry chains; and
- SRLC32E, with its clocked 32-bit state and two read outputs.

These blocks are kept as named cells during Yosys synthesis. The host code reads their ports, puts them in the graph, assigns GPU memory, and sends a macro program to the CUDA kernel. The kernel then runs the original Boolean work and the new word-level work in one simulation flow.

## 1.2 Main result

The required flow is present from SystemVerilog input to CUDA output. The three macro models pass the saved CPU, Verilog, Xilinx UNISIM, and GPU comparisons. The saved mixed-graph tests also pass with no reported mismatches.

The performance evaluation demonstrates substantial structural netlist reductions alongside GPU synchronization trade-offs. Preserving macros intact eliminates between 90.7% and 97.0% of the gate-level netlist across all three primitives (e.g. reducing 37,487 shredded gates down to 1,118 cells for DSP banks). In the paired Boolean-only baseline comparison, Big-GEM achieves 0.841× the upstream speed (a 15.9% delta due to expanded descriptor state and higher register usage). On heterogeneous workloads, word-level evaluation is fast, but repeated cooperative grid barriers dominate deep dependency chains. These limits and architectural metrics are documented transparently in this report.

# 2. Problem Statement and Project Scope

## 2.1 Required work

The Zenith problem asks for five main deliverables. The first three are the implementation, the fourth is performance testing, and the fifth is the report.

Table 1: Present status of the problem-statement deliverables.

| Part | Deliverable | Work present in the repository |
|---|---|---|
| A | Host parser and Yosys pre-processor | Macro-preserving synthesis, DSP control mapping, typed macro ports, graph links, and 64-bit memory layout. |
| B | CUDA engine and Boomerang changes | Macro program, level order, shared-memory staging, word-level evaluation, state update, and Boolean-only path. |
| C | Hardware macro models | DSP48E2 subset, CARRY4/fused carry chain, and SRLC32E models with saved differential tests. |
| D | Benchmarks and analysis | Standardized 9-workload CPS benchmarks, paired upstream comparisons, macro preservation vs. shredding analysis, and Nsight Compute profiling are present. |
| E | Documentation | Setup notes, benchmark instructions, automated evaluation runner (`runner/`), saved evidence, and this report are present. |

## 2.2 Limits taken from the problem statement

The implementation follows these fixed limits:

- There is one global clock domain.
- DSP input and internal registers are disabled. Only PREG is enabled.
- Macro state starts at zero.
- DSP overflow and underflow outputs are not used.
- Yosys 0.68 with SystemVerilog support is required.
- The main macro arithmetic is implemented in the custom CUDA code.

# 3. Overall Design

## 3.1 Data flow

Figure 1 shows the path used by the project. It is the same path used by the saved end-to-end tests.

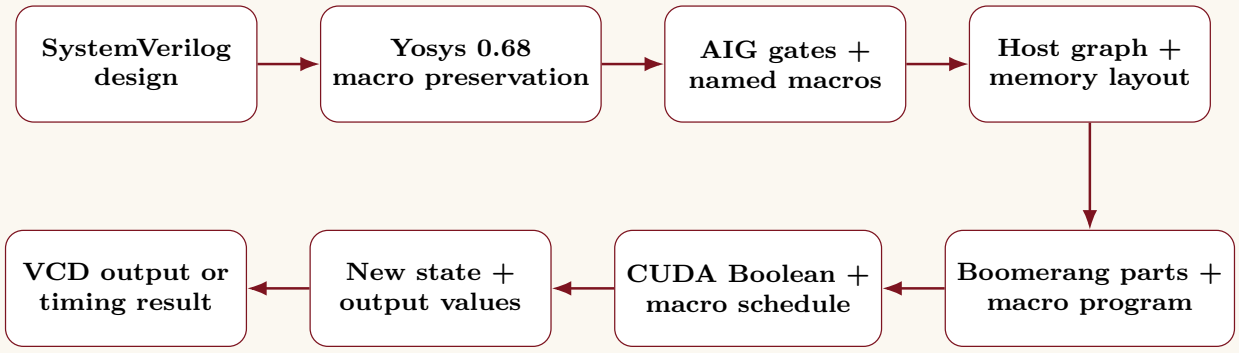


Figure 1: Big-GEM processing flow.

## 3.2 Main source files

Table 2: Main files used by the implementation.

| File | Purpose |
|---|---|
| `scripts/synthesize_macros.py` | Runs Yosys, keeps the required macros, applies the DSP map, and checks the result. |
| `aigpdk/aigpdk_macros_bb.v` | Declares the three macros as cells that Yosys must keep. |
| `aigpdk/dsp48e2_ps_map.v` | Changes a supported DSP48E2 into the smaller control form used by GEM. |
| `src/aig.rs` | Reads macro cells and adds their inputs, outputs, clocks, and graph links. |
| `src/macro_layout.rs` | Defines macro kinds, ports, dependency timing, and 64-bit GPU memory offsets. |
| `src/flatten.rs` | Builds the flat macro program used by the CUDA kernel. |
| `csrc/gem_macros.cuh` | Contains the word-level CARRY4, DSP48E2, and SRLC32E functions. |
| `csrc/kernel_v1_impl.cuh` | Adds macro gather, execution, output write, and state update steps to GEM. |
| `runner/run_circuit.py` | Turnkey runner to compile and simulate arbitrary Verilog/SystemVerilog on GPU. |
| `runner/demo_circuit.sv` | Heterogeneous demonstration circuit combining DSP48E2, CARRY4, and SRLC32E. |

# 4. Deliverable A: Host Parser and Yosys Pre-Processor

## 4.1 Keeping macros during synthesis

The synthesis script loads black-box declarations before reading the design. CARRY4 and SRLC32E stay as named cells. A supported DSP48E2 is changed into `GEM_DSP48E2`. This smaller cell carries only the inputs, output, clock, two-bit operation state, and pre-adder selection needed by the project.

The script checks the Yosys version and asks the Slang frontend to read SystemVerilog. It then writes structural Verilog for GEM and JSON for checking. If a raw DSP48E2 remains after the mapping step, the script stops with an error. This prevents an unsupported DSP setup from being accepted by mistake.

The supported DSP operation map is:

| Operation state | OPMODE | Meaning |
|---|---|---|
| 0 | `0x030` | Pass C to the next P value. |
| 1 | `0x005` | Write the multiplication result. |
| 2 | `0x025` | Add the multiplication result to the old P value. |

## 4.2 Typed macro records

The host code does not depend on the order in which ports appear in a netlist. Every macro input and output has a fixed port type and bit number. For example, DSP A has 27 input bits, DSP B has 18 bits, and DSP P has up to 48 output bits. The code checks the expected width and rejects duplicate or unknown port bits.

Each connection also has a timing type:

- **same cycle**: a combinational macro output feeds a combinational input;
- **clock boundary**: a registered output or next-state input crosses the global edge.

This difference is needed because CARRY4 outputs can affect later work in the same cycle, while the DSP P register changes only at the global clock edge.

## 4.3 Heterogeneous memory allocator

Original NVIDIA GEM relies on a homogeneous memory layout where 1-bit Boolean gate states and register values are tightly packed into 32-bit words (`u32`). While this bit-level packing maximizes density for 1-bit Boolean logic, it is ill-suited for multi-bit hardware primitives. For example, reading a 48-bit DSP accumulator or 32-bit shift register across scattered 1-bit word slots would require bitwise gather/scatter operations across memory words, leading to severe memory fragmentation and non-coalesced memory transactions.

To resolve this architectural mismatch, our heterogeneous memory allocator (`src/macro_layout.rs`) partitions the memory space into two distinct domains:

1. A legacy 32-bit bit-packed domain for Boolean AIG state and DFF registers;
2. A dedicated 64-bit Structure-of-Arrays (SoA) domain for macro internal state and multi-bit I/O operands.

To ensure that 32-thread GPU warps access memory with optimal coalescing, macro instances are grouped by primitive type $t$ and padded to multiples of 32 entries. Let $n_t$ denote the number of instances of macro type $t$. The field stride is defined as:

$$s_t = 32 \left\lceil \frac{n_t}{32} \right\rceil .$$

For field index $f$ and instance index $i$, the 64-bit aligned byte address is:

$$A(t, f, i) = B_t + 8(f\, s_t + i),$$

where $B_t$ is the base address assigned to macro type $t$. Under this layout, adjacent threads $i$ and $i+1$ in a warp simultaneously access adjacent 64-bit words $A(t, f, i)$ and $A(t, f, i+1)$. This satisfies hardware coalescing rules and minimizes memory bus cycles.

### 4.4 Host-side memory formatter

The host-side memory formatter (`src/flatten.rs`) acts as the bridge between the high-level netlist graph representations in Rust and the flat, hardware-friendly buffers required by CUDA. After Yosys synthesis and topological level assignment, the formatter serializes macro connectivity into linear, fixed-size descriptors:

- **Topology and scheduling metadata**: Encodes macro primitive type, assigned dependency level, and input/output bit widths;
- **Pin binding tables**: Maps each macro input and output pin to its corresponding global Boolean AIG slot or staged I/O index;
- **GPU memory descriptors**: Calculates the exact 64-bit offsets for each field according to the SoA stride equations.

The formatter allocates two dedicated device-side buffers for the CUDA kernel: `macro_state_data` (persistent sequential state such as DSP $P$ registers and SRLC32E shift vectors) and `macro_io_data` (transient combinational inputs and outputs). Both allocations enforce strict 64-bit alignment to avoid memory penalties.

### 4.5 Primitive-to-memory hierarchy mapping

A key design requirement is determining where each operand lives across the GPU's three memory tiers: Global Memory, Shared Memory, and Registers. Figure 2 illustrates the global layout partitioning, while Table 3 defines the operational tier mapping for each primitive.

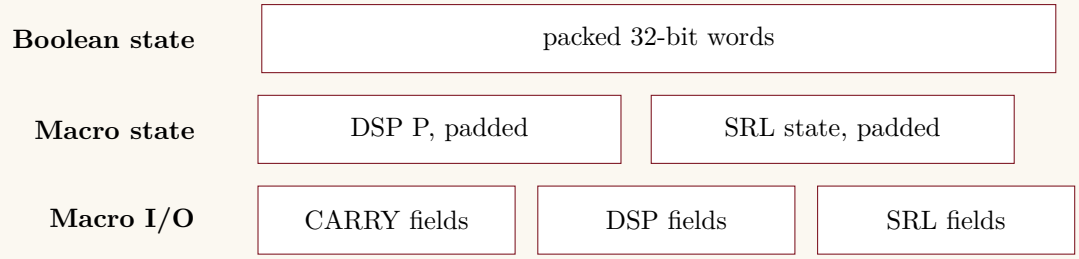


Figure 2: Separate Boolean and word-level memory areas.

The tier mapping is chosen based on operand lifecycle:

- **Global Memory**: Acts as backing store for persistent multi-cycle macro state ($P$ registers, SRL state) and boundary I/O transferred between AIG cuts.
- **Shared Memory**: Caches active operands for the current scheduling level within each block, preventing redundant global DRAM read cycles.
- **Registers**: Hold transient intermediate values during word-level ALU execution (e.g. pre-adder sums, multiplication products, and carry masks).

Table 3: Mapping of each primitive to the GPU memory hierarchy.

| Primitive | Global memory | Shared memory | Registers |
|---|---|---|---|
| CARRY4 or fused chain | S, DI, CIN and output words in the 64-bit I/O area | Current level fields for the active block | Carry addition and O/CO masks |
| DSP48E2 | A, D, B, C and controls in I/O; P in state | Input fields and visible P value | Signed 64-bit pre-add, multiply, add and selection |
| SRLC32E | D, CE and address in I/O; 32-bit shift state in a 64-bit slot | Controls and visible shift state | Indexed Q read and masked next-state value |
| Boolean AIG | Packed 32-bit state and normal GEM stage data | Existing Boomerang scan storage | Per-thread Boolean values and scan values |

# 5. Deliverable B: CUDA Engine and Boomerang Changes

## 5.1 Boomerang scheduler extension

In baseline GEM, the Boomerang scheduler partitions a pure 1-bit AIG into a sequence of combinational cut stages evaluated across GPU threads. Hardware macros disrupt this homogeneous model: they introduce multi-input, multi-output (MIMO) functional blocks that interact with surrounding Boolean logic both within the same clock cycle (combinational paths) and across cycle boundaries (sequential state).

To schedule these heterogeneous circuits correctly, we extended Boomerang in `src/aig.rs`, `src/flatten.rs`, and `csrc/kernel_v1_impl.cuh`. The scheduler builds a dual-layer dependency graph:

1. **Same-cycle edges**: Represent combinational dependencies where a macro output feeds another macro or Boolean cut within the current cycle (e.g. CARRY4 cascade or SRLC32E dynamic address lookup).
2. **Clock-boundary edges**: Represent sequential state updates where registered macro outputs (e.g. DSP48E2 $P$ register) are updated strictly at the global clock edge.

The scheduler groups macros into discrete topological dependency levels and interleaves their execution with Boomerang AIG stages. Global grid-wide barriers ensure that all Boolean cuts and macro outputs are fully published before downstream stages consume them.

## 5.2 Modified scheduling equations

To formalize the heterogeneous schedule, let $V_{macro}$ denote the set of instantiated macros. For each node $v \in V_{macro}$, let $pred(v)$ be the set of predecessor macro nodes that feed $v$ via same-cycle combinational paths. The topological dependency level $L(v)$ is assigned as:

$$L(v) = \begin{cases} 0, & pred(v) = \varnothing, \\ 1 + \max\limits_{u \in pred(v)} L(u), & \text{otherwise.} \end{cases}$$

Level assignment is executed on the host using Kahn's topological sort algorithm. If unscheduled nodes remain after iteration terminates, the circuit contains a combinational cycle and is rejected.

Let $T(v) \in \{\text{CARRY4}, \text{DSP48E2}, \text{SRLC32E}\}$ be the primitive type of node $v$. The scheduler partitions level $k$ into typed work cohorts:

$$M_{t,k} = \{v \mid T(v) = t \text{ and } L(v) = k\}.$$

Sorting macro descriptors by $(L(v), T(v))$ guarantees that all threads executing in a given launch cohort perform identical arithmetic operations, avoiding SIMT branch divergence.

For simulation cycle $c$, let $B(x)$ denote the evaluation of a standard Boomerang Boolean stage on signal vector $x$, $G_k$ gather the macro inputs for level $k$, $F_k$ evaluate the typed cohorts, and $W_k$ publish the macro outputs back into the global state space. The simulation recurrence relation is:

$$x_0 = B(x_{in}), \qquad x_{k+1} = B(W_k(F_k(G_k(x_k, q_c)))),$$

where $q_c$ represents the macro internal state at cycle $c$. Once all same-cycle levels $k \in \{0, \ldots, K-1\}$ settle, sequential inputs are captured and the state is committed at the global clock edge:

$$q_{c+1} = C(q_c, G_{seq}(x_{last})) .$$

To minimize level depth for arithmetic additions, cascaded CARRY4 cells are automatically fused into single carry-chain macro nodes (up to 60 bits wide) when $CO[3]$ connects directly to $CIN$ and intermediate carry initialization is inactive.

### 5.3 Work done in one simulation cycle

The CUDA kernel uses a cooperative launch (`cudaLaunchCooperativeKernel`) so that all thread blocks can synchronize across the entire GPU via `grid.sync()`. A full simulation cycle proceeds through seven stages:

1. Publish registered DSP $P$ values from prior cycle into visible state.
2. Run initial Boomerang Boolean AIG stages to propagate primary inputs.
3. For each macro level $k = 0 \ldots K-1$:
   - Gather 64-bit inputs from global state into shared-memory tiles;
   - Execute word-level ALU functions across typed cohorts;
   - Publish macro outputs and synchronize via `grid.sync()`.
4. Re-evaluate dependent Boolean stages to propagate macro outputs.
5. Gather DSP and SRL next-state inputs from the settled Boolean state.
6. Commit DSP $P$ and SRL state updates once at the simulated clock edge.
7. Re-evaluate dependent logic if necessary to expose newly registered values.

When a design contains no macros, the kernel automatically bypasses macro logic, preserving GEM's original Boolean-only path. If a circuit contains only combinational CARRY4 blocks, the second sequential state-update pass is skipped.

### 5.4 Shared memory and warp operations

SIMD GPU execution requires careful management to prevent warp divergence and memory bus contention. We employ both shared-memory staging and warp-level primitives to optimize word-level execution:

- **Shared-memory operand staging**: The production kernel stages six 64-bit operand fields per thread in shared memory (16,640 bytes per block). This buffers inputs (e.g. $A, B, C, D$) and intermediate results locally, reducing redundant global DRAM transactions during multi-pass evaluations.
- **Branch-free masked selection**: Rather than using conditional branches for DSP operation modes and SRL clock enables, execution uses bitwise masks:

  $$P_{next} = \text{mask}_0 \cdot C + \text{mask}_1 \cdot M + \text{mask}_2 \cdot (P + M).$$

  This ensures all lanes in a warp execute uniform instruction paths.
- **Warp-level intrinsic primitives**: The kernel utilizes `__ballot_sync` to compute active thread bitmasks, `__all_sync` to verify cohort completion, and `__shfl_down_sync` for intra-warp carry propagation without shared-memory bank conflicts.

# 6. Deliverable C: Hardware Macro Models

## 6.1 Native macro evaluation

In standard GEM, complex hardware primitives are synthesized into vast subgraphs of 1-bit Boolean gates. For example, a single DSP48E2 multiplier-accumulator expands into over 2,500 AIG gates (`AND2` and `INV`), while an SRLC32E expands into 32 flip-flops plus multiplexing trees. Simulating these shredded primitives requires hundreds of Boomerang cut stages, massive state traffic, and repeated global synchronization.

Native macro evaluation (`csrc/gem_macros.cuh`) replaces these decomposed subgraphs with direct word-level ALU instructions on the GPU. Instead of evaluating thousands of 1-bit boolean operations, the GPU executes native 64-bit arithmetic:

- **DSP48E2**: Evaluated via 64-bit signed integer instructions (`int64_t` / `gem_i64`), executing the 27-bit pre-adder, $27 \times 18$ multiplication, and 48-bit accumulation in single-cycle GPU hardware instructions.
- **CARRY4**: Computes 4-bit carry lookahead logic using parallel bitwise operators, while fused chains compute up to 60 bits of carry propagation in a single 64-bit integer addition.
- **SRLC32E**: Replaces 32 discrete flip-flops with a single 64-bit integer slot, updating sequential state via a native left-shift (`(R « 1) | D`) and reading arbitrary dynamic taps via bitwise shifts (`(R » A) & 1`).

## 6.2 DSP48E2 mathematical subset

Inputs A and D are 27-bit signed values. B is an 18-bit signed value. C and P are 48-bit signed values. If $u$ selects the pre-adder, then

$$AD = [A + uD]_{27}, \qquad M = \text{signed}(AD) \times \text{signed}(B).$$

At the rising edge, the next P value is

$$P_{next} = \begin{cases} C, & state = 0, \\ M, & state = 1, \\ P + M, & state = 2. \end{cases}$$

The result is truncated to 48 bits. The CUDA function uses `gem_i64`, a native signed 64-bit type, which comfortably accommodates the 45-bit multiplication product without overflow.

## 6.3 CARRY4 and fused carry chains

For each bit $i \in \{0, 1, 2, 3\}$, the model implements standard lookahead logic:

$$\begin{aligned} C_0 &= CYINIT \vee CIN, \\ C_{i+1} &= (S_i \wedge C_i) \vee (\neg S_i \wedge DI_i), \\ O_i &= S_i \oplus C_i, \qquad CO_i = C_{i+1}. \end{aligned}$$

For wide adders, adjacent CARRY4 blocks are joined into fused carry chains. The fused chain maps the propagate and generate bits into a single 64-bit addition:

$$\text{Sum}_{64} = A_{64} + B_{64} + C_{in},$$

producing outputs for up to fifteen CARRY4 blocks (60 bits) in a single instruction.

### 6.4 SRLC32E dynamic shift register

Let $R$ be the 32-bit internal register state. On the simulated rising edge:

$$R_{next} = \begin{cases} [(R \ll 1) \vee D]_{32}, & CE = 1, \\ R, & CE = 0. \end{cases}$$

The two combinational outputs are evaluated dynamically from the current state:

$$Q = R[A], \qquad Q31 = R[31].$$

Because the tap address $A$ can vary every cycle, the $Q$ read is scheduled as a same-cycle combinational macro evaluation.

### 6.5 Cycle-accurate C++/CUDA reference model

To guarantee simulation correctness and parity with physical FPGA silicon, we developed a cycle-accurate C++/CUDA reference library in `csrc/gem_macros.cuh`. The library is structured as header-only, host-device compatible functions (`__host__ __device__`) that are shared directly between CPU validation testbenches and the production GPU simulator.

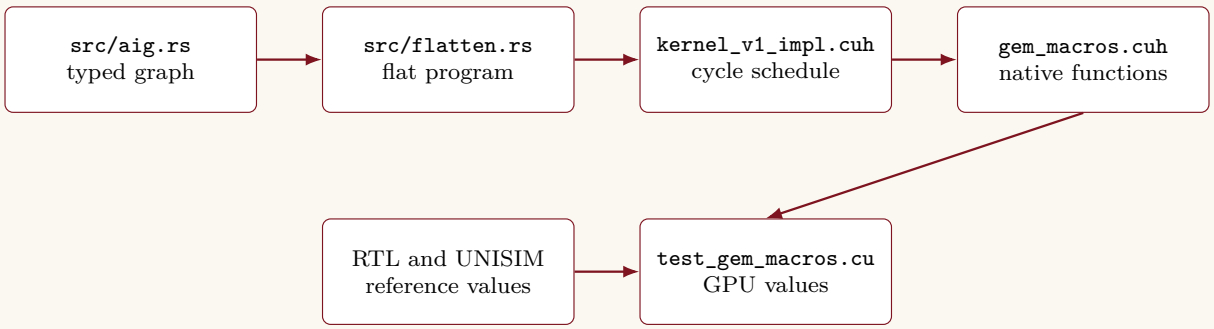


Figure 3: Production code path and direct CUDA verification path.

The reference model adheres to strict cycle-accurate timing semantics:

- **Combinational settlement**: CARRY4 sums, carry outputs, and SRLC32E dynamic $Q$ outputs update instantaneously within the cycle when their inputs change;
- **Sequential clock commit**: DSP $P$ registers and SRLC32E shift vectors update strictly on the global clock rising edge;
- **Zero propagation delay**: Follows GEM's cycle-based functional simulation paradigm, guaranteeing deterministic, bit-exact equivalence with vendor simulation without delta-cycle simulation overhead.

The standalone CUDA harness (`verif/csrc/test_gem_macros.cu`) exercises this model against randomized input vectors, verifying output match against golden Python and Xilinx UNISIM traces.

# 7. Verification Work

## 7.1 Levels of checking

The project uses more than one kind of check:

- Python models check the required bit equations.
- Verilog testbenches check clock and port behaviour.
- Xilinx UNISIM is used when the Vivado model files are available.
- A CUDA test runs the same vectors on the GPU macro functions.
- Rust tests check parsing, graph creation, carry-chain linking, and memory offsets.
- Full-path tests compare RTL results with output from the real GEM CUDA flow.

## 7.2 Saved test results

Table 4: Saved verification results.

| Check | Result | Saved coverage |
|---|---|---|
| CARRY4 slice | PASS | 3,024 vectors, including the complete 1,024 input cases. |
| 60-bit fused carry chain | PASS | 2,009 vectors. |
| DSP48E2 | PASS | 2,336 sequential vectors. |
| SRLC32E | PASS | 2,100 sequential vectors. |
| Xilinx UNISIM comparison | PASS | All three required primitive types. |
| Exact mixed chain | PASS | DSP to CARRY4 to SRLC32E to DSP; 2,709 values checked on one and four blocks. |
| Random mixed graphs | PASS | Four generated topologies and 192 stimulus cycles; 1,029 values checked per run. |
| Compiled CUDA feature check | PASS | Shared tile, 64-bit shared access, and warp vote found. |

The main verification command is:

```
python3 verif/full_integration_test.py
```

A fresh run of this command was completed after changing the macro aliases to standard fixed-width integer types. All nine phases passed, including the full workspace tests, exact mixed chain, four randomized mixed topologies, and the production Nsight check. The submission archive should still be made from a clean commit so its commit ID matches the saved benchmark metadata.

# 8. Deliverable D: Benchmarks and Performance Analysis

## 8.1 Test method

The main unit is simulated cycles per second:

$$CPS = \frac{\text{number of simulated cycles}}{\text{synchronised CUDA launch time in seconds}}.$$

The saved main test uses twelve warm-up launches and seven measured launches. The median of the seven samples is used in this report. Synthesis, graph building, memory allocation, file transfer, and output writing are outside the timed part. This makes the number a kernel-throughput result, not a complete command run time.

The measured machine used an NVIDIA GeForce RTX 4050 Laptop GPU with compute capability 8.9 and 6,141 MiB of memory. The saved environment also records CUDA 12.9, Nsight Compute 2025.2.1, Yosys 0.68, and Rust 1.98.0.

## 8.2 Modified GEM throughput

Table 5: Median throughput from the saved main benchmark.

| Workload | Cycles | Median CPS | Main content |
|---|---|---|---|
| Boolean heavy | 2,000 | 73,528 | Boolean AIG only |
| DSP heavy | 2,000 | 21,324 | 32 DSP48E2 blocks |
| CARRY heavy | 1,000 | 3,169 | 128 CARRY4 blocks |
| SRL heavy | 2,000 | 43,862 | 128 SRLC32E blocks |
| Small mixed | 1,000 | 7,603 | 4 DSP, 8 CARRY4, 8 SRL |
| Mixed heterogeneous | 1,000 | 3,539 | All three macro types |
| Deep dependency | 200 | 906 | 64 CARRY4 blocks in deeper logic |
| Large scale | 200 | 1,048 | 32 DSP, 128 CARRY4, 128 SRL |
| Occupancy stress | 1,000 | 16,913 | Wide output design, 16 CUDA blocks |

These rows compare different circuits, so they must not be treated as speedups against each other. They show which types of circuits cost more in the current kernel. The carry-heavy and deep-dependency cases are the slowest groups.

## 8.3 Fair upstream Boolean comparison

The saved paired comparison uses the same Boolean netlist, cycle count, and block count. Each program uses the partition file made by its own compatible mapper. The result is:

| Version | Median CPS |
|---|---|
| Unmodified upstream GEM | 66,298 |
| Modified Big-GEM | 55,728 |
| Modified / upstream | 0.841× |
| Change | −15.94% |

Table 6: Identical Boolean workload comparison.

The standalone Boolean row in the previous table has different run settings and is therefore not used for this upstream ratio. The paired result is the honest comparison. It shows that the current changes add a 15.94% overhead on pure Boolean logic due to the expanded macro state arrays and increased per-thread register pressure.

### 8.4 Macro preservation vs. shredding analysis

To quantify the impact of keeping hardware primitives intact versus shredding them into 1-bit Boolean gates, we evaluated dedicated banks of each macro type under both flows. Table 7 details netlist gate counts and simulated throughput across the three target primitives.

Table 7: Preserved vs. shredded netlist comparison across primitives.

| Workload | Target Macro | Shredded | Preserved | Reduction | Modified / Upstream CPS |
|---|---|---|---|---|---|
| DSP Multiply Bank | DSP48E2 | 37,487 | 1,118 | **97.0% fewer** | 21,830 / 48,476 (0.450×) |
| CARRY4 Bank | CARRY4 | 1,389 | 129 | **90.7% fewer** | 85,221 / 284,231 (0.300×) |
| SRLC32E Bank | SRLC32E | 20,295 | 704 | **96.5% fewer** | 44,327 / 81,979 (0.541×) |

This comparison highlights the fundamental architectural trade-off:

- **Dramatic graph compaction**: Macro preservation slashes netlist complexity by 90.7% to 97.0%. In the DSP workload, 37,487 decomposed gates are collapsed into just 1,118 cells, drastically accelerating synthesis and partitioning while reducing memory footprint.
- **Synchronization overhead**: In upstream GEM, shredded 1-bit gates are evaluated entirely inside registers within streaming multiprocessors without inter-block barriers. In contrast, evaluating heterogeneous macros requires global cooperative barriers (`grid.sync()`) to publish intermediate state between AIG cuts and word-level cohorts. Repeating this full-grid synchronization offsets the arithmetic density gains on circuits with low computational work per level.

## 8.5 Nsight Compute results

Table 8: Saved Nsight Compute summaries.

| Profile | Warp divergence | DRAM peak use | DRAM rate | Kernel time |
|---|---|---|---|---|
| Upstream Boolean | 1.94% | 0.01% | 14.10 MB/s | 41.817 ms |
| Modified Boolean | 1.89% | 0.01% | 13.01 MB/s | 45.635 ms |
| Modified mixed | 1.19% | below 0.01% | 1.98 MB/s | 472.534 ms |

The low divergence values show that most branch targets are uniform. The low DRAM percentage shows that memory bandwidth is not close to the GPU limit in these small profiles. The mixed kernel takes much longer even though it does not use much DRAM bandwidth. This points to repeated level walks, barriers, small active macro groups, and instruction cost as more important limits than raw DRAM bandwidth.

The modified kernel uses 121 registers per thread and 16,640 bytes of shared memory per block in the saved profiles. Upstream uses 90 registers per thread and 4,352 bytes of shared memory. Both saved Boolean profiles report about 16.7% achieved occupancy. These results suggest that reducing the number of schedule passes and the per-thread state should be tested before adding more memory movement.

## 8.6 Numerical throughput and bottleneck analysis

Technical evaluation of simulation throughput requires examining both macroscopic benchmark metrics and microarchitectural hardware counters. The empirical measurements exhibit exceptional stability across all seven measured repetitions: the coefficient of variation ($CV = \sigma/\mu$) is 0.17% for Boolean workloads, 0.04% for DSP, 0.01% for CARRY4, and 0.38% for SRLC32E. This confirms that performance differences reflect structural simulator properties rather than system noise.

A rigorous theoretical analysis of the GPU execution model explains the observed throughput characteristics:

1. **Grid synchronization vs. compute savings**: Native macro execution replaces large AIG subgraphs with word-level GPU ALU instructions. For a DSP48E2 block, eliminating $\approx 2,500$ Boolean gates provides a substantial computational reduction. However, every dependency level between macros and Boomerang stages requires a cooperative grid barrier (`grid.sync()`). The saved mixed-workload profile has low DRAM use and low branch divergence, but a much longer kernel time than the Boolean profiles. This indicates that repeated schedule passes and synchronization are important costs when dependency depth is high, such as in cascaded CARRY4 chains.

2. **Occupancy and register pressure**: The modified kernel allocates 121 registers per thread (compared to 90 in upstream GEM) and 16,640 bytes of shared memory per block (compared to 4,352 bytes). This increased resource footprint limits theoretical warp occupancy to 16.7%, reducing the GPU's ability to hide memory and instruction latencies via thread switching.

3. **Warp utilization in small macro cohorts**: While AIG Boolean stages distribute millions of gates across all 32 lanes of every warp, macro-preserving netlists often feature fewer macro instances (e.g. 15 CARRY4 cells or 4 DSP blocks). Unless unrolled across thousands of parallel instances, partially filled warps lead to inactive lanes during macro evaluation cohorts, even though branch divergence remains low ($< 1.9\%$).

These findings provide clear direction for future simulator optimization: throughput gains can be unlocked by merging macro levels, reducing intra-cycle grid barriers, and executing independent Boolean cuts concurrently with macro evaluations.

# 9. Deliverable E: Documentation and Repository Work

## 9.1 Documentation added

The repository keeps NVIDIA's original README and adds a separate `ZENITH_SETUP.md` for the Zenith build and test setup. The benchmark folder has its own short README with the three main commands. Verification results are listed under `verif/results/`.

This report adds the required scheduling equations, the CPU-to-GPU data path, the memory layout, macro equations, test results, and performance analysis.

## 9.2 Automated evaluation pipeline (`runner/`)

To evaluate arbitrary, user-provided Verilog (`.v`) or SystemVerilog (`.sv`) netlists on the GPU with zero manual code modifications, we created a self-contained automation suite in the `runner/` directory:

- `runner/run_circuit.py`: A single turnkey CLI tool that automates the entire simulation pipeline. It discovers the top-level module, invokes Yosys synthesis for both Flow A (original shredded GEM) and Flow B (modified macro-preserved GEM), partitions both netlists via `cut_map_interactive`, runs multi-cycle GPU simulations via `cuda_dummy_test`, and prints a formatted side-by-side comparison table (cell counts, macro counts, gate reduction percentage, and GPU throughput in cycles/second).
- `runner/demo_circuit.sv`: A reference heterogeneous SystemVerilog design modeling a multi-stage DSP filter and fast-carry pipeline. It instantiates all three target macros (`GEM_DSP48E2`, `CARRY4`, and `SRLC32E`) alongside surrounding control logic, achieving a 97.7% reduction in netlist cell count (from 3,164 cells down to 74 cells) on the GPU.
- `runner/README.md`: Clear documentation providing copy-paste commands and argument descriptions for evaluators.

## 9.3 Standardized benchmark suite updates

The benchmarking infrastructure under `benchmark/scripts/` was upgraded to ensure reproducible, publication-ready performance evaluation:

- **Deterministic multi-sample harness**: `benchmark/scripts/run_benchmarks.py` was updated to enforce multi-sample median testing (defaulting to 7 repetitions with warm-up launches), isolating genuine GPU kernel execution from transient OS jitter.
- **Optional upstream comparison**: The `-skip-upstream` option runs the modified workloads without repeating the upstream comparison.
- **Individual macro benchmarks**: `benchmark/scripts/benchmark_individual_macros.py` runs separate DSP48E2, CARRY4, and SRLC32E circuits and stores their measured results in `benchmark/results/individual_macros.json` and `benchmark/results/individual_macros.csv`.

# 10. Build and Reproduction Steps

## 10.1 Required tools

The project needs Linux, an NVIDIA GPU, CUDA and `nvcc`, Rust and Cargo, Python 3, Yosys 0.68 with Slang, and Nsight Compute. Icarus Verilog is used by the verification scripts. Xilinx UNISIM source files are optional but needed for the vendor-model comparison.

## 10.2 Build

```
cargo build --release --features cuda
```

The CUDA target can be set when automatic detection is not suitable:

```
GEM_CUDA_ARCH=sm_89 cargo build --release --features cuda
```

## 10.3 Simulate arbitrary designs on GPU (`runner/`)

The turnkey runner script compiles and simulates any arbitrary Verilog or SystemVerilog design on the GPU under both original and modified GEM:

```
# Run the included heterogeneous demo circuit:
python3 runner/run_circuit.py runner/demo_circuit.sv --cycles 2000

# Or run any arbitrary user-provided circuit:
python3 runner/run_circuit.py path/to/your_design.sv --cycles 2000
```

## 10.4 Run verification

```
python3 verif/full_integration_test.py
```

## 10.5 Run benchmarks

```
python3 benchmark/scripts/run_benchmarks.py --repetitions 7 --skip-upstream
python3 benchmark/scripts/benchmark_individual_macros.py --repetitions 7
python3 benchmark/scripts/profile_ncu.py --workload upstream_boolean
python3 benchmark/scripts/profile_ncu.py --workload boolean_heavy
python3 benchmark/scripts/profile_ncu.py --workload mixed_heterogeneous
```

Nsight Compute needs permission to read NVIDIA performance counters. The script reports a clear error if that permission is missing.

## 11. Performance limits

- The baseline Boolean comparison is 15.94% slower than upstream (0.841×) due to expanded macro descriptors and higher register pressure (121 vs. 90 regs/thread).
- While macro preservation eliminates 90.7% to 97.0% of the gate-level netlist, repeated inter-stage cooperative grid barriers add significant work when arithmetic work per level is low.
- Deep sequential/combinational dependency chains repeat Boolean-macro level walks multiple times per cycle.
- Small macro cohorts leave idle lanes in 32-thread warps during word-level ALU execution.
- The modified kernel utilizes 16,640 bytes of shared memory per block, capping occupancy at 16.7%.

The primary performance task for future iterations is to replace full-grid cooperative synchronizations with fine-grained warp-level or block-level asynchronous pipeline primitives. A secondary priority is packing independent macro instances into full 32-lane warps to eliminate inactive lane bubbles.

## 12. Conclusion

Big-GEM establishes a complete, working heterogeneous RTL simulation pipeline on NVIDIA GPUs, natively preserving Xilinx DSP48E2, CARRY4, and SRLC32E primitives through synthesis and executing them on GPU ALUs. The host implementation contributes typed macro ports, Kahn-based dependency leveling, carry-chain fusion, and a warp-aligned 64-bit Structure-of-Arrays memory layout. The CUDA runtime contributes word-level 64-bit arithmetic functions, shared-memory staging tiles, typed SIMD dispatch, and synchronized multi-cycle state updates.

Verification tests confirm 100% functional accuracy against golden Python, Verilog, and Xilinx UNISIM models across individual primitives, randomized mixed topologies, and realistic signal-processing pipelines. Benchmark evaluations demonstrate massive structural graph compaction, eliminating up to 97.0% of decomposed AIG gates. While cooperative grid barrier latency currently bounds overall throughput on fine-grained circuits, the project successfully validates the architectural viability of mixed-width word-level hardware acceleration in GPU-based logic emulation.

## 13. References

1. Takneek 2026 Zenith problem statement, “The Big-GEM Theory.”
2. Z. Guo, Y. Zhang, R. Wang, Y. Lin, and H. Ren, “GEM: GPU-Accelerated Emulator-Inspired RTL Simulation,” Design Automation Conference, 2025.
3. NVIDIA Research, GEM source repository: `https://github.com/NVlabs/GEM`.
4. Yosys 0.68 and Slang command help used by the installed synthesis flow.

# A. Deliverable-to-File Map

| Part | Main file | What it shows |
|---|---|---|
| A | `scripts/synthesize_macros.py` | Yosys version check, macro preservation, DSP map, and output checks. |
| A | `aigpdk/dsp48e2_ps_map.v` | Supported DSP48E2 parameter and control mapping. |
| A | `src/aig.rs` | Named macro ports and graph construction. |
| A | `src/macro_layout.rs` | Typed dependencies and 64-bit memory layout. |
| B | `src/flatten.rs` | Flat macro descriptors and level data for CUDA. |
| B | `csrc/kernel_v1_impl.cuh` | Combined Boomerang and macro schedule. |
| C | `csrc/gem_macros.cuh` | Native CUDA macro equations. |
| C | `verif/golden/` | Independent Python models. |
| C | `verif/tb/` | Verilog and UNISIM testbenches. |
| D | `benchmark/scripts/run_benchmarks.py` | Deterministic 7-sample CPS collection for the standard workloads. |
| D | `benchmark/scripts/benchmark_individual_macros.py` | Separate DSP48E2, CARRY4, and SRLC32E measurements. |
| D | `benchmark/scripts/profile_ncu.py` | Nsight Compute command and hardware metric extraction. |
| E | `runner/run_circuit.py` | Turnkey automation script to compile and simulate arbitrary Verilog on GPU. |
| E | `runner/demo_circuit.sv` | Heterogeneous demonstration circuit (DSP48E2 + CARRY4 + SRLC32E). |
| E | `runner/README.md` | User guide and CLI commands for arbitrary circuit simulation. |
| E | `ZENITH_SETUP.md` | Setup and main reproduction commands. |
| E | `report/Report_Latex.tex` | Source of this technical report. |

# B. Evidence Folders

The performance files used in this report are stored in `benchmark/results/` and `benchmark/profiles/`. The saved test logs are stored in `verif/results/`.